

4.1 Saving Data

Objectives

Saving key-value pairs of simple data types in a shared preferences file

Saving arbitrary files in Android's file system

Using databases managed by SQLite

Using NoSQL - Firebase

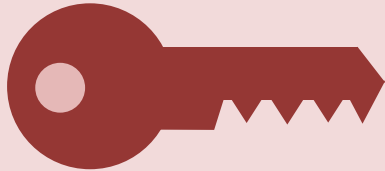
ViewModel, Repository, LiveData and Coroutine

Saving Data

Shared Preferences

Private or Public

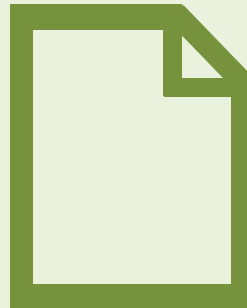
Key-value pair



Data Files

Private or Public

Data file; text, sound, images, and etc

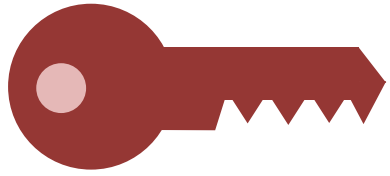


SQLite

Private

Repeating and structured data





Shared Preferences

Stores private primitive data in key-value pairs

Primitive data: boolean, float, int, long, and string

Data will persist across session, even if an app is killed

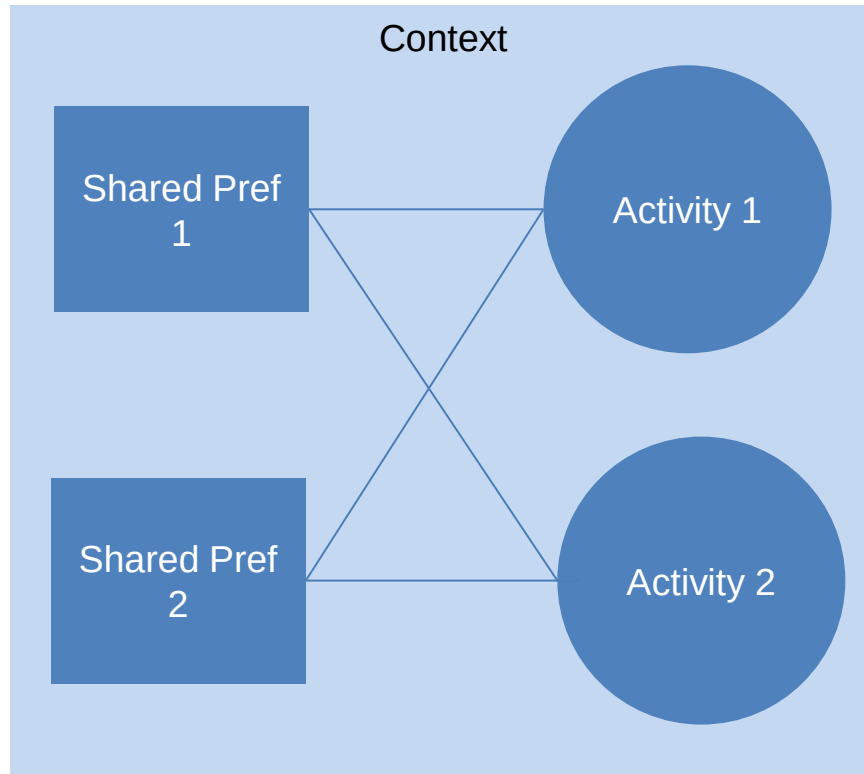
Use the SharedPreferences class

Shared Preferences

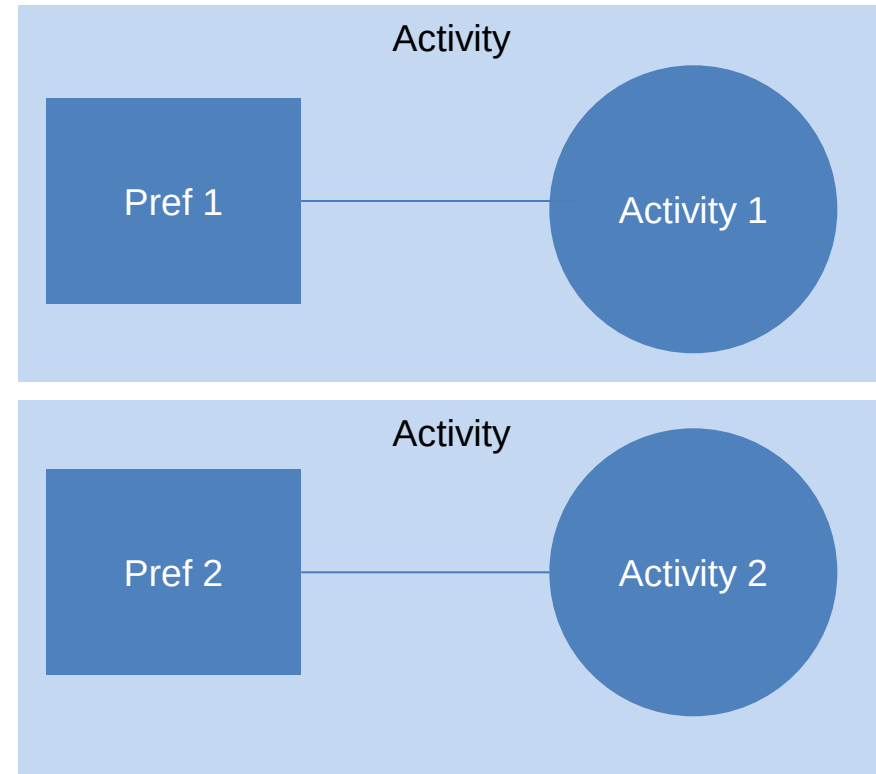
Method	getSharedPreferences	getPreferences
Number of shared preference file	Multiple	Single
Access level	Context	Activity

Shared Preferences

Shared Preferences



Preference



Jetpack Compose – DataStore

Jetpack compose uses DataStore instead of SharedPreferences.

```
dependencies {  
    implementation "androidx.datastore:datastore-preferences:1.0.0"  
}
```

Jetpack Compose – DataStore

- **val Context.dataStore**

This is an **extension property** for the Context class, meaning you can call context.dataStore anywhere in your app.

- **DataStore<Preferences>**

This specifies that we are using Preferences DataStore, which is a key-value storage system (similar to SharedPreferences).

- **by preferencesDataStore(name = "settings")**

This is **delegated property initialization**,

```
val Context.dataStore by preferencesDataStore(name = "settings")

@Composable
fun SettingsScreen(context: Context) {
    val scope = rememberCoroutineScope()
    val datastore = context.dataStore
    val exampleKey = stringPreferencesKey("example_key")

    var value by remember { mutableStateOf("") }

    // Read value from DataStore
    LaunchedEffect(Unit) {
        value = datastore.data.first()[exampleKey] ?: "Default Value"
    }
}
```

Jetpack Compose – DataStore

- Writing Data

```
val exampleKey = stringPreferencesKey("example_key")

suspend fun saveData(value: String, context: Context) {
    context.dataStore.edit { preferences ->
        preferences[exampleKey] = value
    }
}
```

Jetpack Compose – DataStore

- **Reading Data**

```
val exampleKey = stringPreferencesKey("example_key")

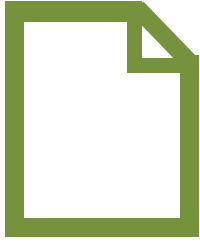
val readData: Flow<String> = context.dataStore.data
    .map { preferences -> preferences[exampleKey] ?: "Default Value" }
```

- `.edit {}` → Modifies and saves data asynchronously.
- `.map {}` → Reads data as a Flow (asynchronously).

Question 4.1.1?

1. What is the main difference between the `getPreferences` and `getSharedPreferences` methods?
2. A `SharedPreferences` file can be accessed by:
 - a. multiple Activities
 - b. multiple apps





Saving Files

A File APIs for reading and writing large amount of data

Suitable for image files or anything exchanged over a network

The exact location of the where your files can be saved might vary across devices

Internal and External Storage

Characteristics	Internal	External
Removeable	No	Yes (Possible)
Availability	Always	Not always
Accessibility	Your app	World-readable

Save a file on internal storage

An app's internal storage directory is specified by the app's package name

No permission is required to perform read/write operations

Use `getFreeSpace()` or `getTotalSpace()` to check available internal storage space to avoid IO Error

Create and write a file

```
// Create a new file
val file = File(context.filesDir, filename)

// Write to a file
file.bufferedWriter().use { out ->
    out.write(fileContent)
    out.close()
}

// Read from a file
if(file.exists()){
    val content = file.bufferedReader().readText()
    file.bufferedReader().close()
}
```

External Storage

Removable storage media (such as an SD card) or an internal (non-removable) storage

Files saved to the external storage are:

- world-readable
- can be modified by the user
- can become unavailable if the user unmount or remove the storage (SD Card)

External Storage

- Request permission:

```
<manifest ...>
  <!-- If you need to modify files in external storage, request
        WRITE_EXTERNAL_STORAGE instead. -->
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="18" />
</manifest>
```

- Two types of files:

Characteristics	Public	Private
Accessibility	All apps	Specific app*
State after uninstallation of app	Available	Deleted

*Files stored in an *app-specific directory*—accessed using [Context.getExternalFilesDir\(\)](#).

Verify external storage

```
/* Checks if external storage is available to at least read */  
fun isExternalStorageReadable(): Boolean {  
    return Environment.getExternalStorageState() in  
        setOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)  
}
```

Saving Private File

```
fun getPrivateAlbumStorageDir(context: Context, albumName: String): File? {  
    // Get the directory for the app's private pictures directory.  
    val file = File(context.getExternalFilesDir(  
        Environment.DIRECTORY_PICTURES), albumName)  
  
    if (!file?.mkdirs()) {  
        Log.e(LOG_TAG, "Directory not created")  
    }  
    return file  
}
```

Note: Other directories include DIRECTORY_MUSIC, DIRECTORY_PICTURES, DIRECTORY_RINGTONES and etc

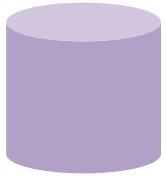
Storage Guide (New)

Type	Content	Other apps can access?	Files removed on app uninstall?
App-specific files	Files means for your app's use only	No	Yes
Media	Shareable media files (images, audio files, videos)	Yes (with permission)	No
Documents and other files	Other types of shareable content, including downloaded files	Yes	No
App preferences	Key-value pairs	No	Yes
Database	Structure data	No	Yes

Question 4.1.2

1. Among internal and external storage, which one is suitable for files that you want to allow user to access with a computer?
2. All Android devices offered build-in non-volatile memory that is non-removeable but can be accessed with a computer. Is this part classified as internal storage?





What is SQLite?



Find out more from www.sqlite.org

SQLite Data Type

Type	Definition
NULL	The value is a NULL value.
INTEGER	The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
REAL	The value is a floating point value, stored as an 8-byte
TEXT	The value is a text string
BLOB	Binary large object. Suitable for images, audio or other media objects

SQLite Database

SQLite can be accessed using the [Room persistence library](#)

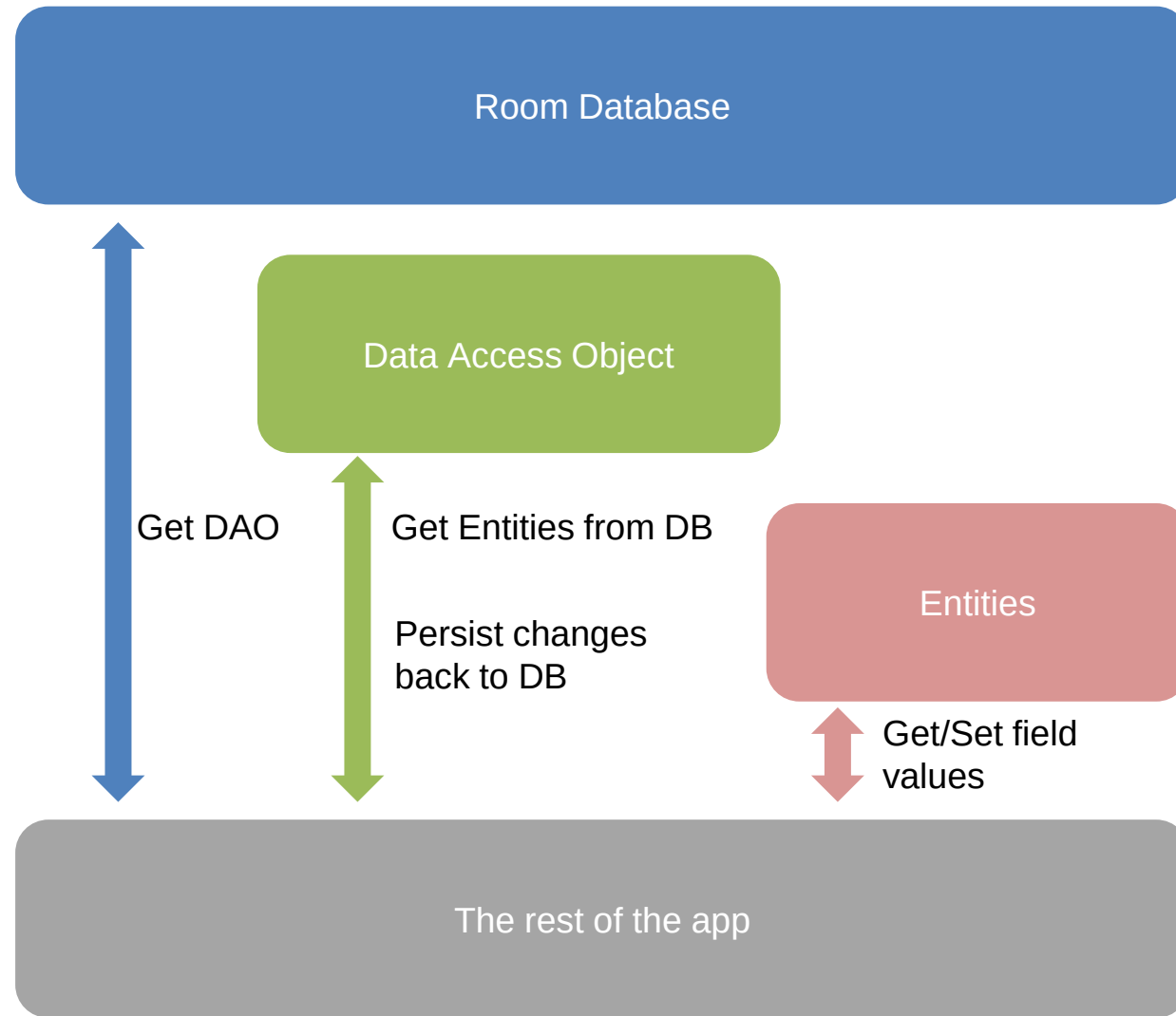
3 components:

- Database: access point to DB
- Entity: table
- Data access object (DAO): methods to access DB

Using the Room

Dependencies for Room should be included in the `build.gradle` file

```
dependencies {  
    def room_version = "2.3.0"  
  
    implementation "androidx.room:room-runtime:$room_version"  
  
    // For Kotlin use kapt instead of annotationProcessor  
    annotationProcessor "androidx.room:room-compiler:$room_version"  
    // optional - Kotlin Extensions and Coroutines support for Room  
    implementation "androidx.room:room-ktx:$room_version"  
  
    // optional - RxJava support for Room  
    implementation "androidx.room:room-rxjava2:$room_version"  
  
    // optional - Guava support for Room, including Optional and ListenableFuture  
    implementation "androidx.room:room-guava:$room_version"  
  
    // Test helpers  
    testImplementation "androidx.room:room-testing:$room_version"  
}
```



Entity

Step 1: Create an entity class for each table

```
@Entity
data class User(
    @PrimaryKey val uid: Int,
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)
```

A nullable attribute



Entity

```
// An entity with a composite primary key
@Entity(tableName = "users" primaryKeys = arrayOf("firstName", "lastName"))

// Define foreign key constraints between entities
@Entity(foreignKeys = arrayOf(ForeignKey(
    entity = User::class,
    parentColumns = arrayOf("id"),
    childColumns = arrayOf("user_id"))
    )
)
```

DAO

Step 2: Create a DAO

```
@Dao
interface UserDao {
    @Query("SELECT * FROM user")
    fun getAll(): List<User>

    @Query("SELECT * FROM user WHERE uid IN (:userIds)")
    fun loadAllByIds(userIds: IntArray): List<User>

    @Query("SELECT * FROM user WHERE first_name LIKE :first AND " +
        "last_name LIKE :last LIMIT 1")
    fun findByName(first: String, last: String): User

    @Insert
    fun insertAll(vararg users: User)

    @Delete
    fun delete(user: User)
}
```



Variable number of arguments

AppDatabase Abstract Class

Step 3: Create an AppDatabase abstract class

```
@Database(entities = arrayOf(User::class), version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

Instance

Step 4: Create a database instance

```
val db = Room.databaseBuilder(  
    applicationContext,  
    AppDatabase::class.java, "database-name"  
).build()
```

Using Room

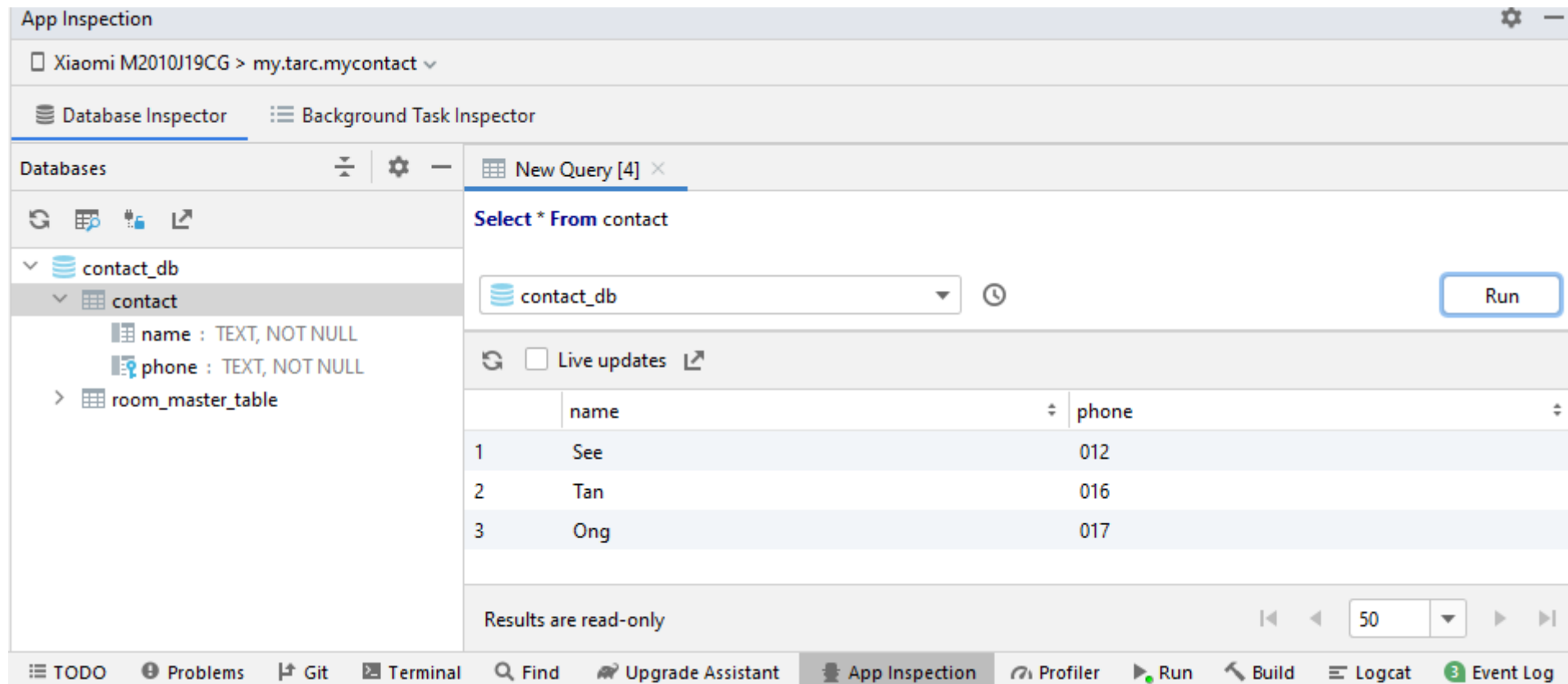
Don't use Room for database access on the UI thread.
Asynchronously run the query on a background thread

More information about background thread will be covered in
Chapter 4.2

- Example of using Room:
<https://github.com/seekweeteck/Demo4.1>

Database Inspector

Inspect, query, and modify your app's databases



Question 4.1.3

1. Explain the purpose of the following objects:
 - a. Entity
 - b. Data Access Object (DAO)
 - c. Room



Question 4.1.3

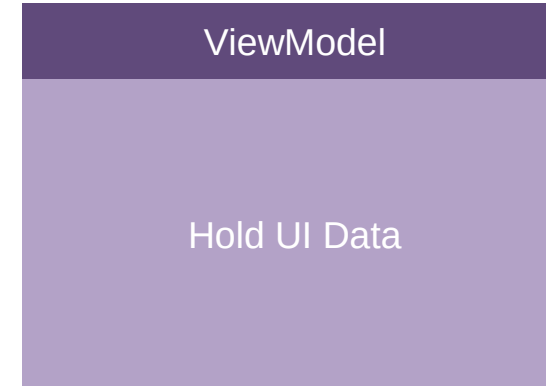
An online property agency, myhome.com.my, is looking for a mobile solution to make its service accessible to more agents. The company would like to create a mobile app which allows its agents to upload media files, post comments, and record locations of properties. Explain TWO techniques to be used to implement the above-mentioned system.

ViewModel, Repository, LiveData and Coroutine

ViewModel

Provide data for UI components

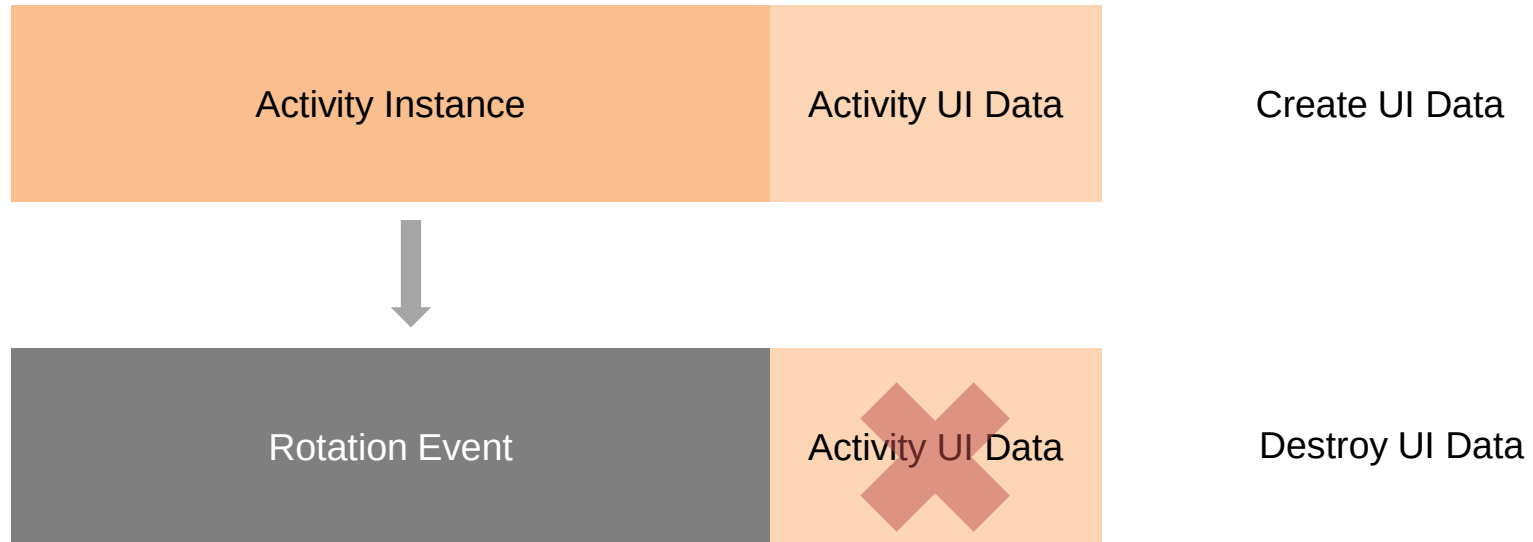
It survive configuration changes



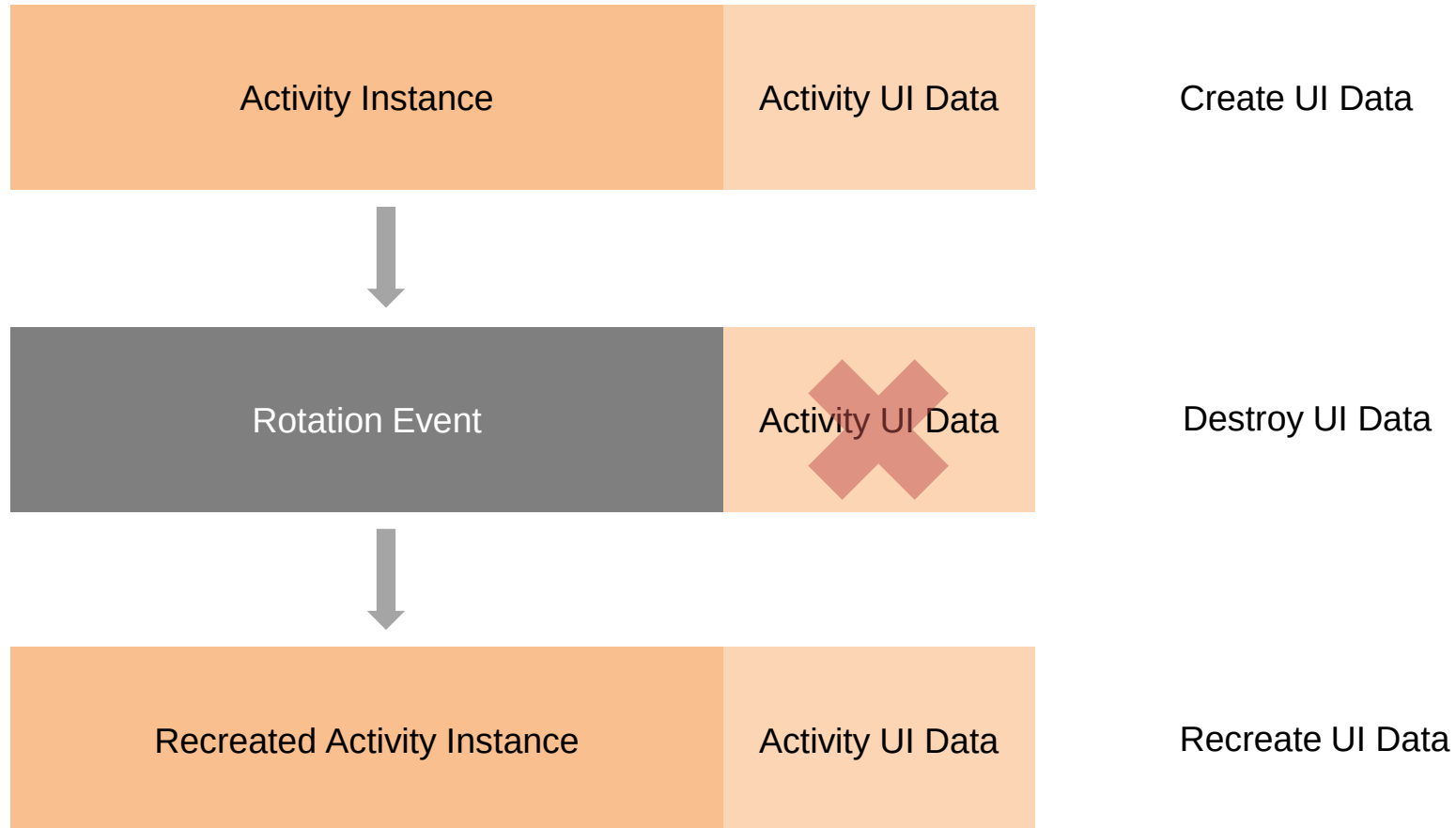
Traditional



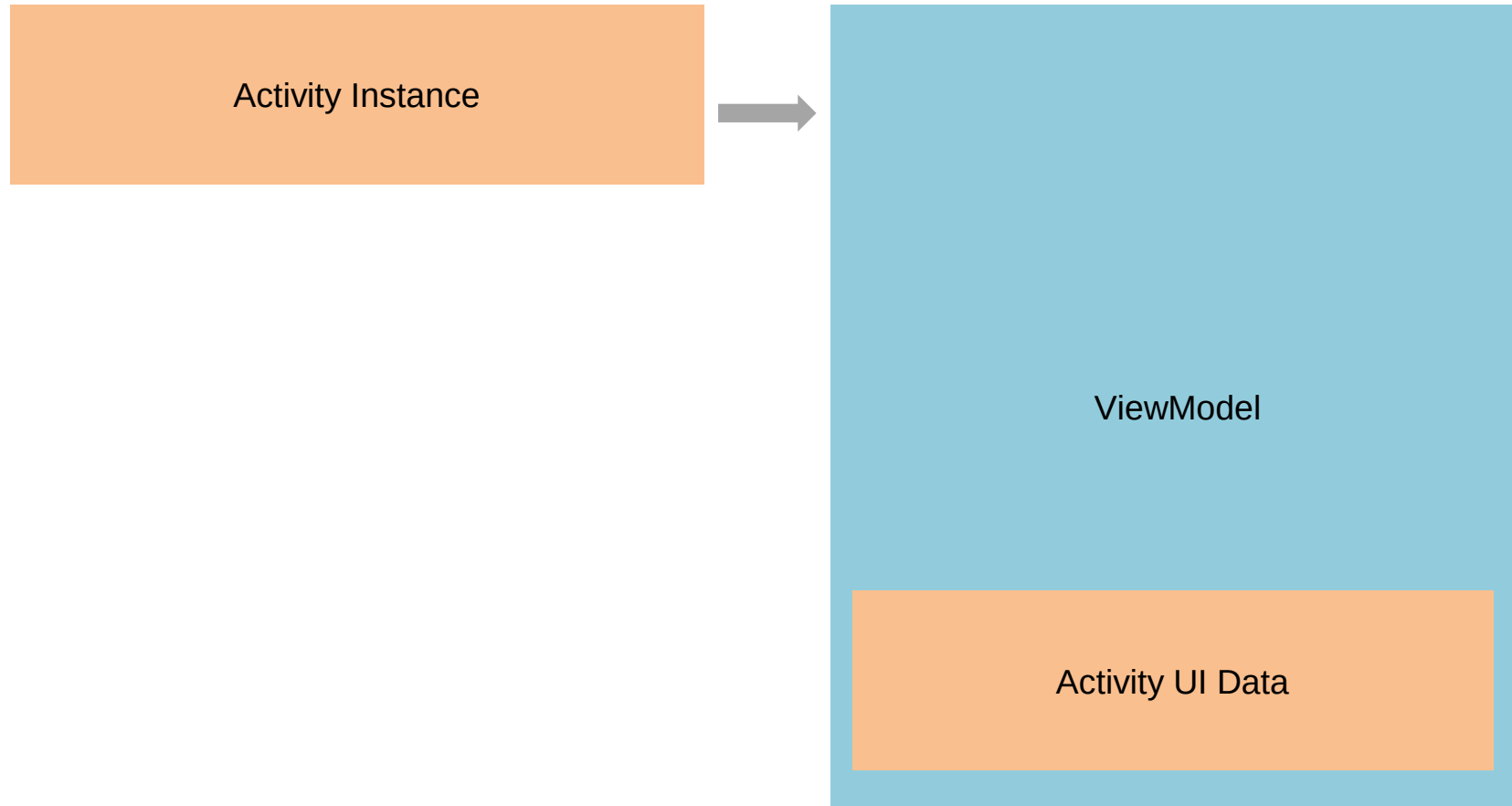
Traditional



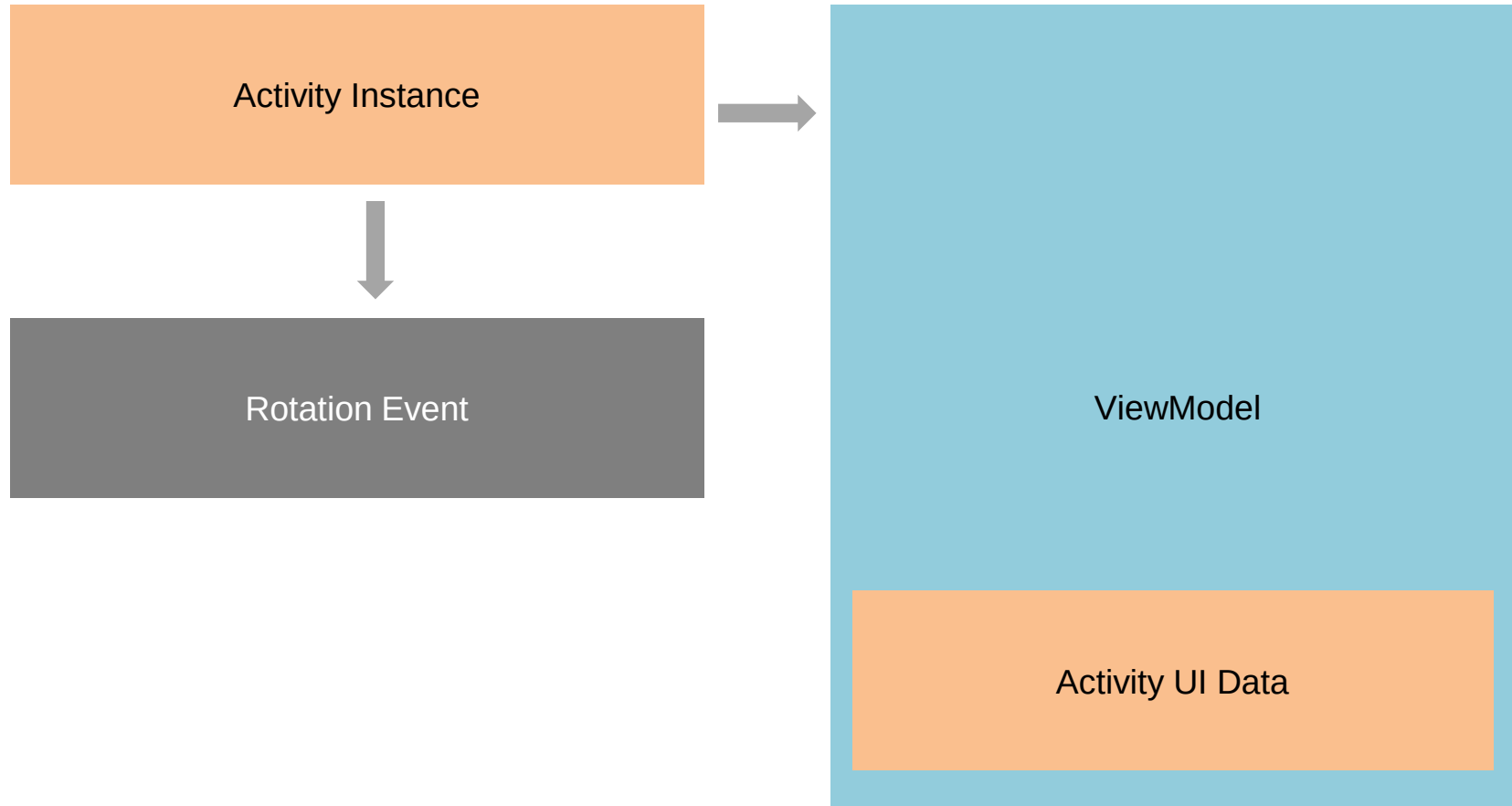
Traditional



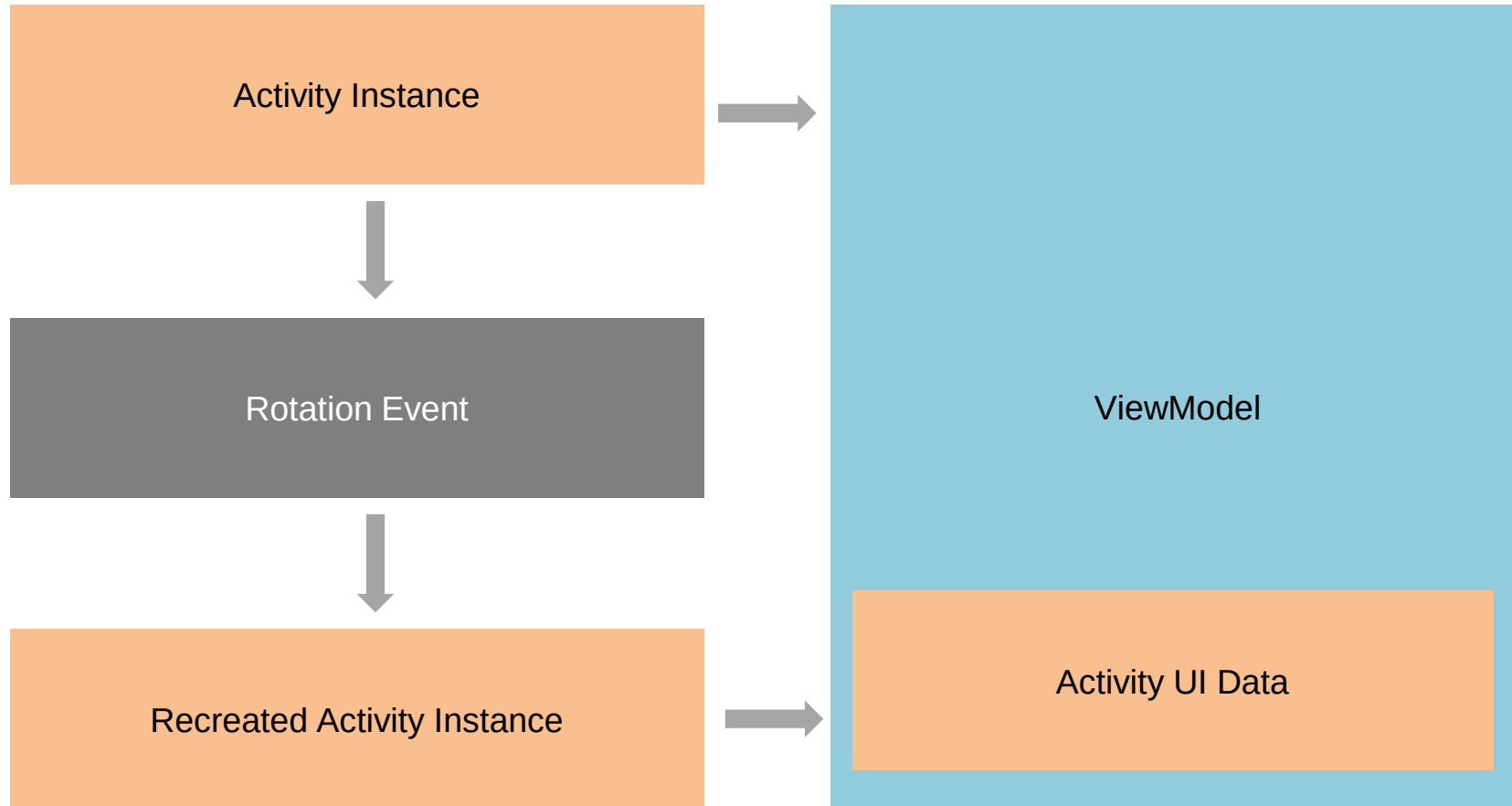
ViewModel

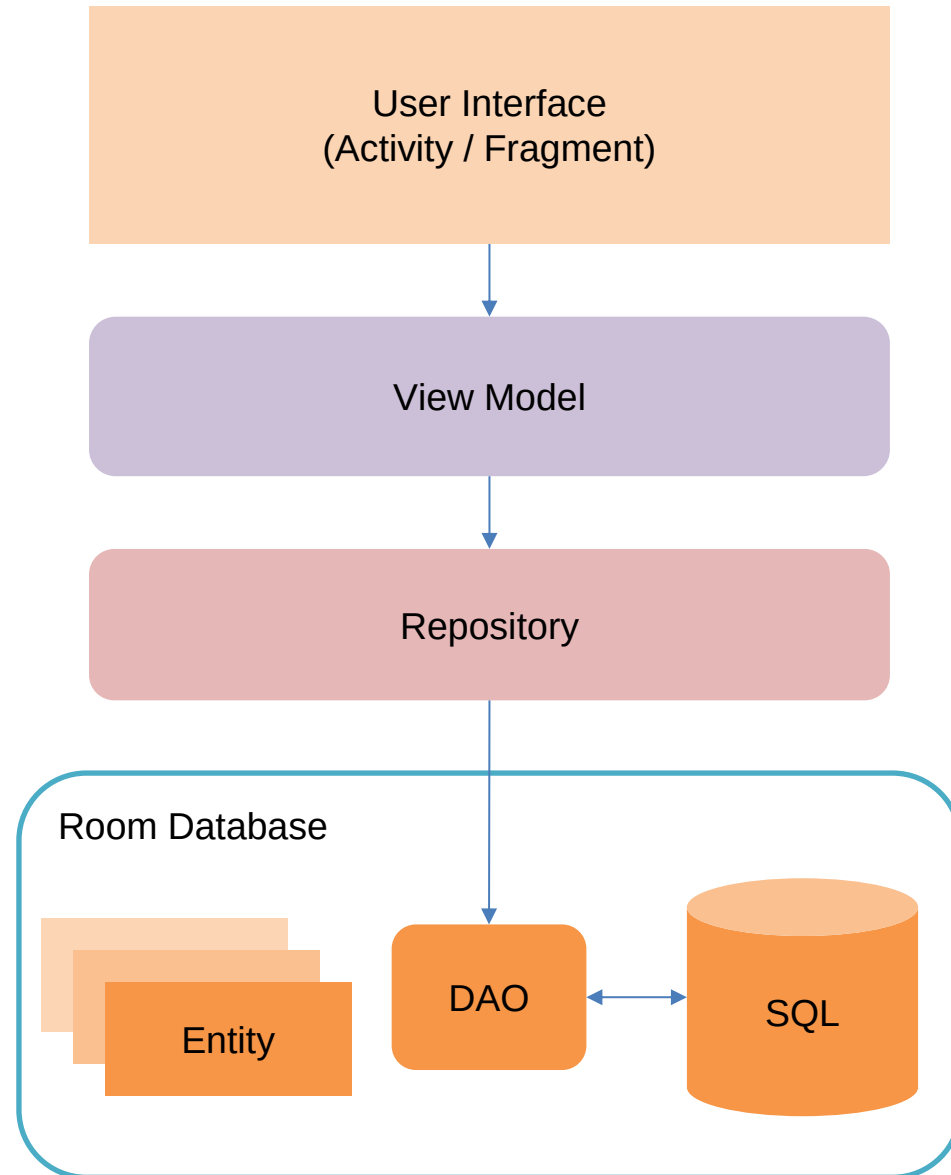


ViewModel



ViewModel





Repository

It provides access to multiple data sources

Manages query threads and allows you to use multiple backends

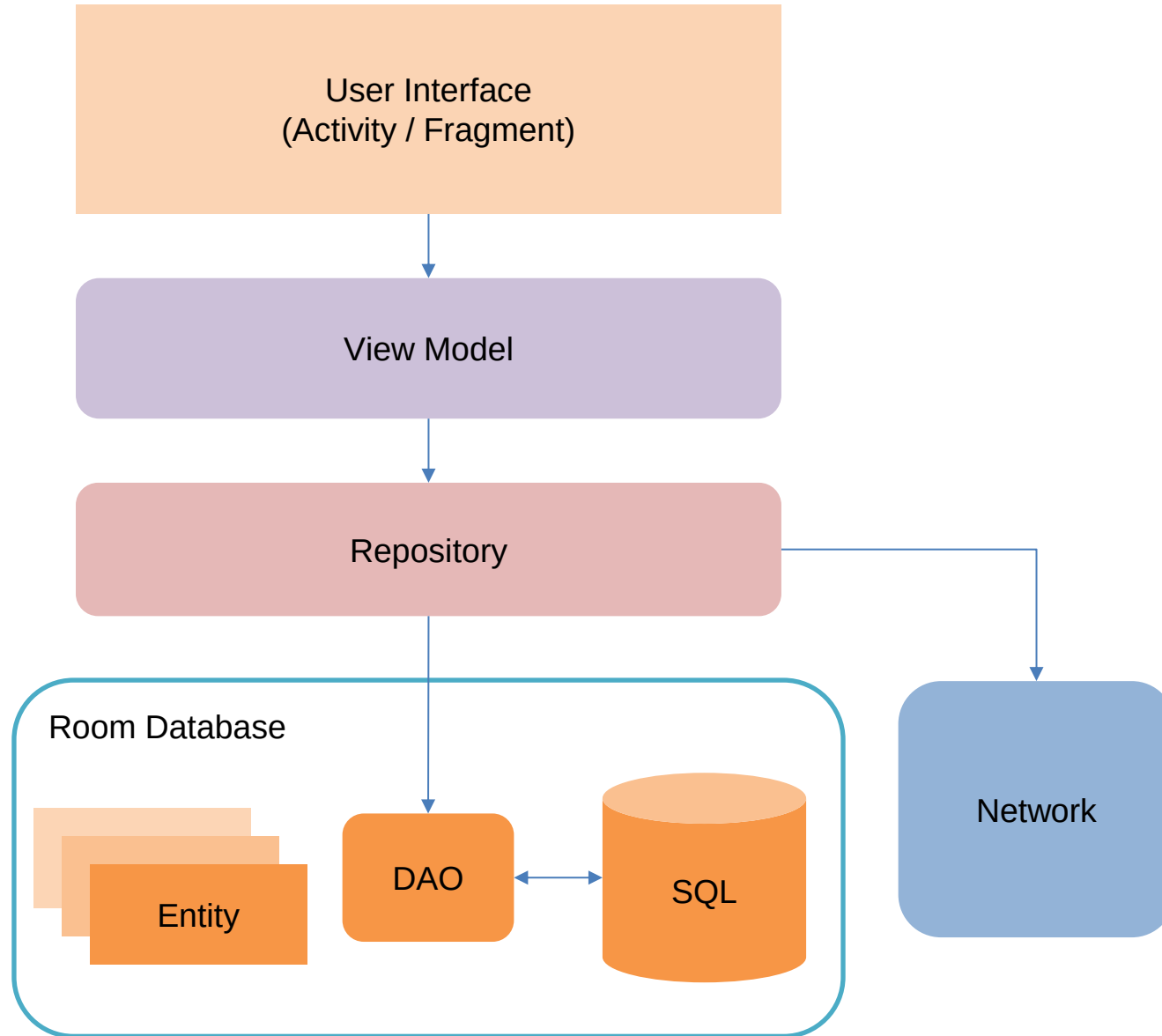
E.g. A Repository implements the logic for deciding whether to fetch data from a network or use results cached in a local database

Entity, DAO, and SQLite

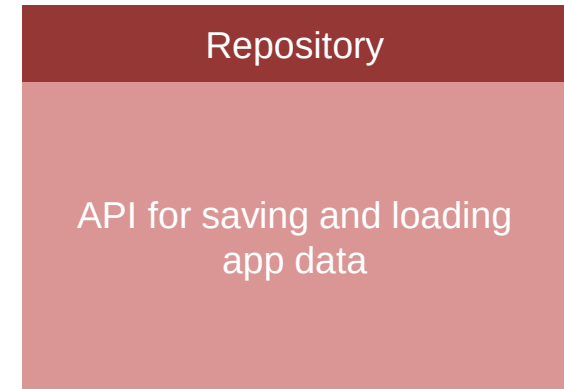
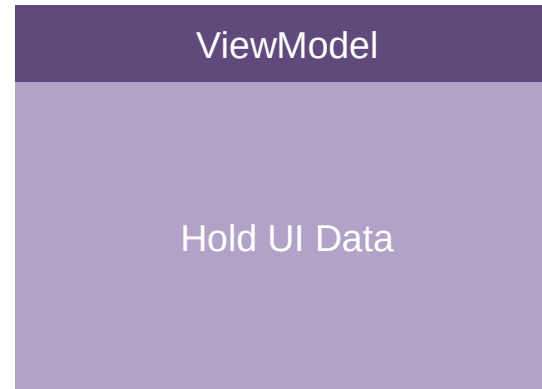
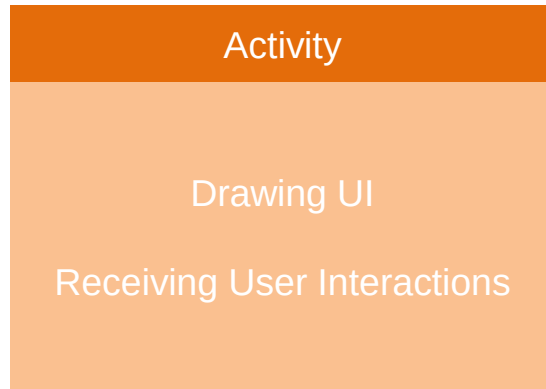
Entity = a table

DAO = Data access object provides methods to interact with data

SQLite = a relational database for mobile



Activity, ViewModel, and Repository



Question 4.1.4

1. What is the purpose of the following components?
 - a. Activity
 - b. ViewModel
 - c. Repository



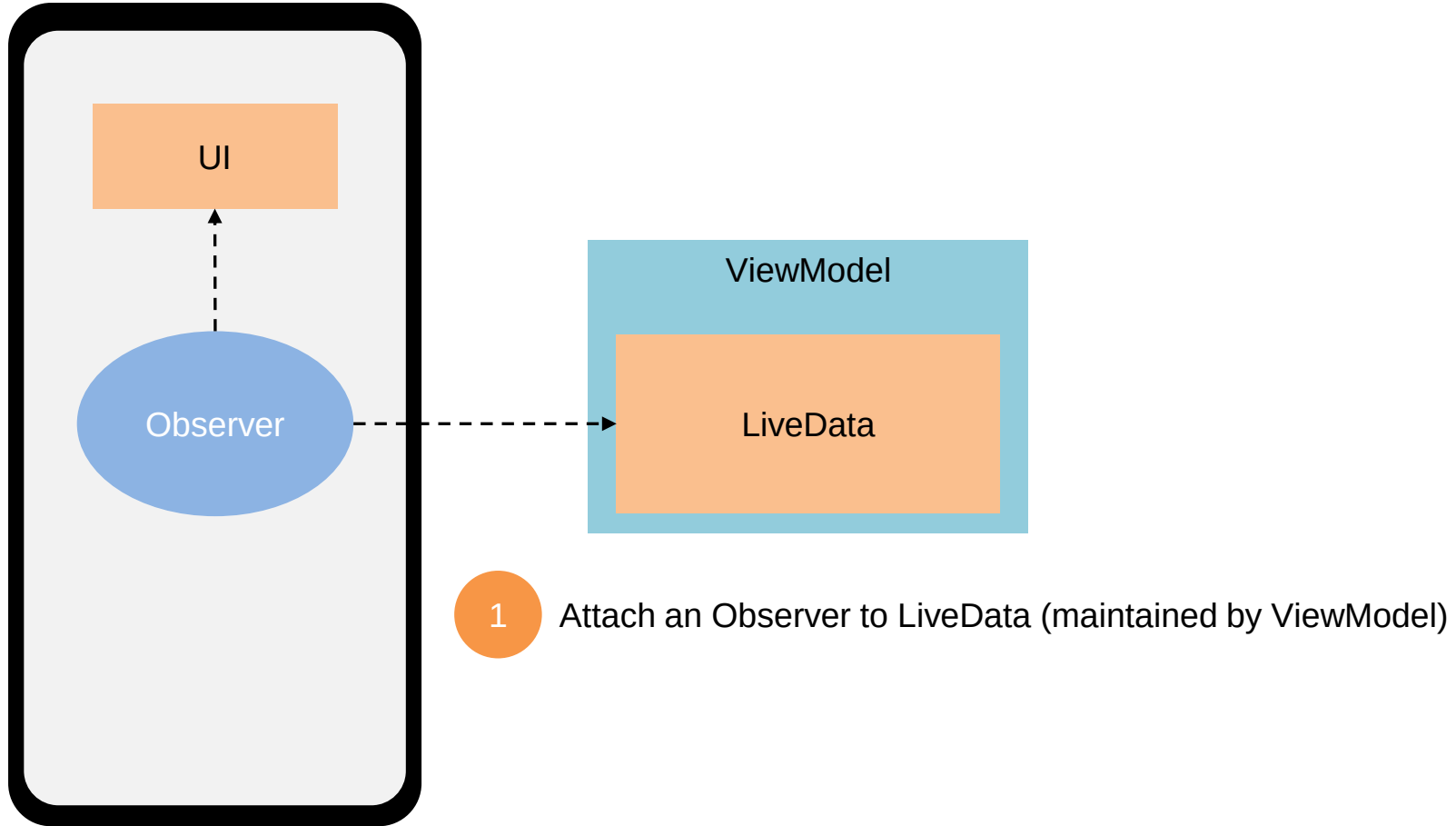
LiveData

An observable data holder class

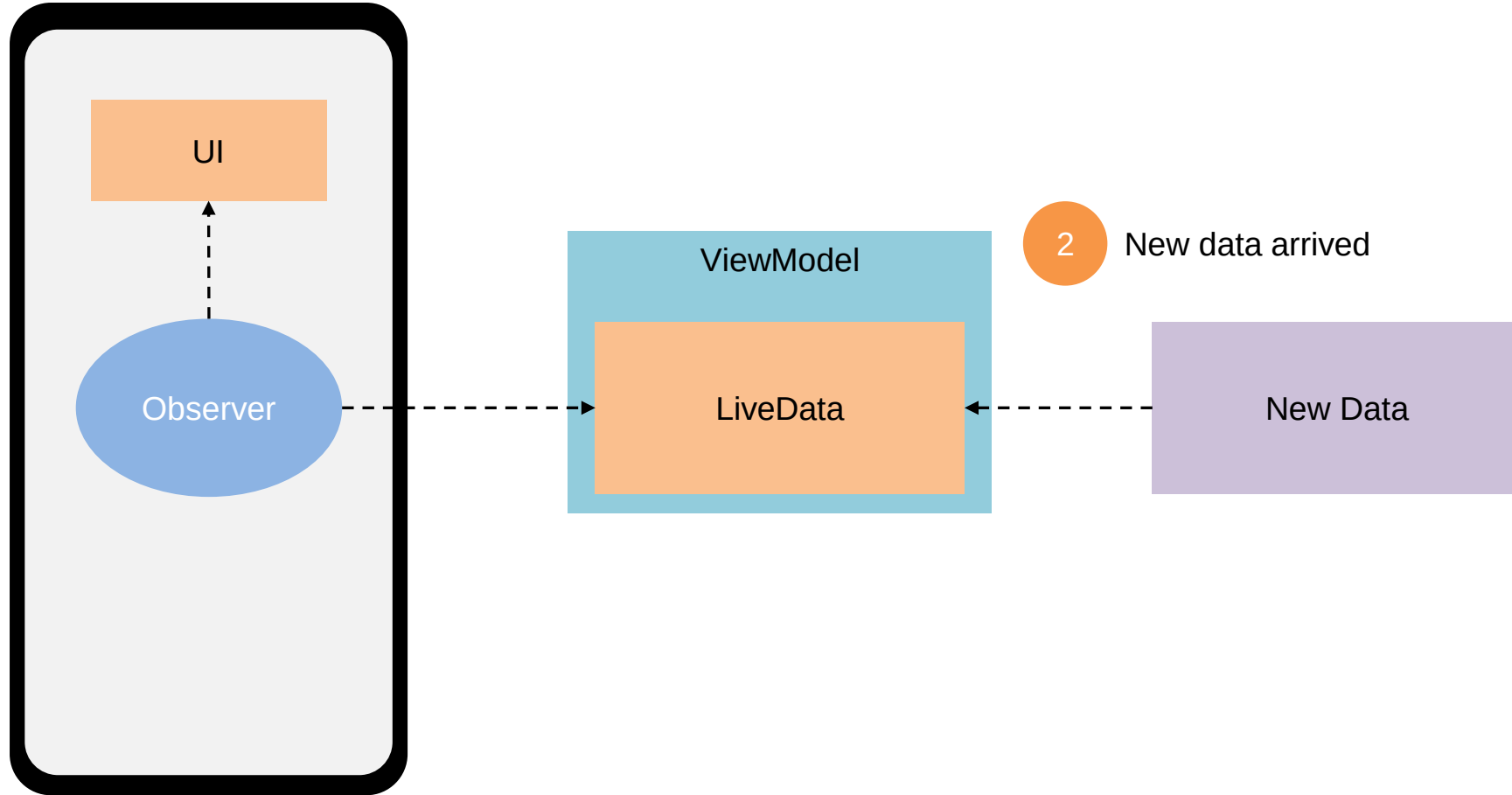
An observer is notified when data held by LiveData object changes

It is lifecycle-aware

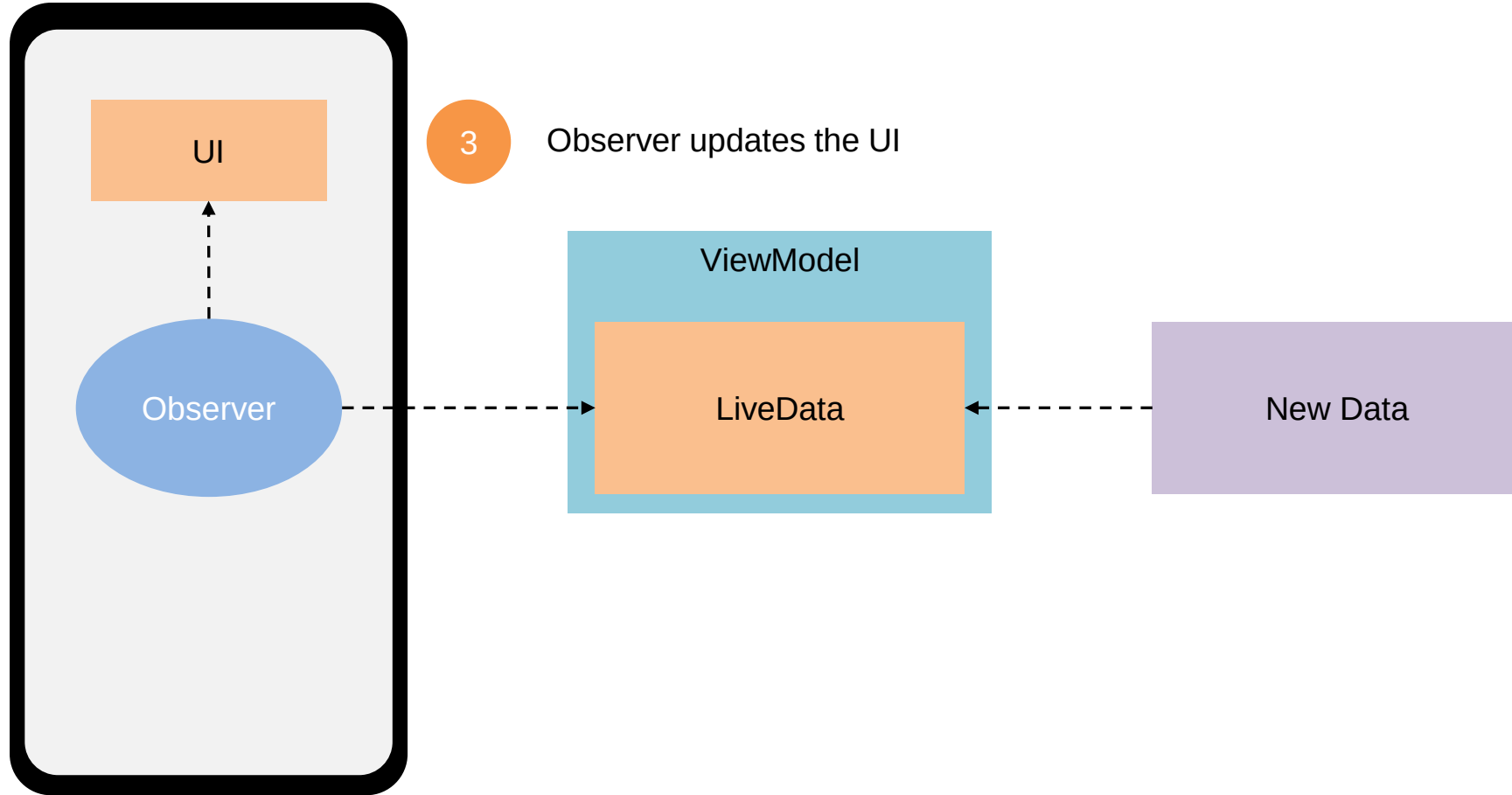
LiveData



LiveData



LiveData



```
class UserProfileViewModel: ViewModel() {  
    private val _user = MutableLiveData<User>()  
  
    val user : LiveData<User>  
        get() = _user  
}
```

```
override fun onCreate(savedInstanceState: Bundle?){  
    ...  
    userViewModel.user.observe(this,  
        Observer{  
            user -> userNameTextView.text = user?.name  
        }  
    )  
}
```

```
class UserProfileViewModel: ViewModel() {  
    private val _user = MutableLiveData<User>()  
  
    val user : LiveData<User>  
        get() = _user  
}
```

```
override fun onCreate(saveInstanceState: Bundle?){  
    ...  
    userViewModel.user.observe(this,  
        Observer{  
            user -> userNameTextView.text = user?.name  
        }  
    )  
}
```

```
class UserProfileViewModel: ViewModel() {  
    private val _user = MutableLiveData<User>()  
  
    val user : LiveData<User>  
        get() = _user  
}
```

```
override fun onCreate(savedInstanceState: Bundle?){  
    ...  
    userViewModel.user.observe(this,  
        Observer{  
            user -> userNameTextView.text = user?.name  
        }  
    )  
}  
  
...  
user.setValue(newUser)
```



Jetpack Compose StateFlow

StateFlow is a **hot, observable data stream** designed for **state management** in Kotlin Coroutines. Unlike LiveData, it is **fully compatible with Jetpack Compose** and is the **recommended approach** for managing UI state.

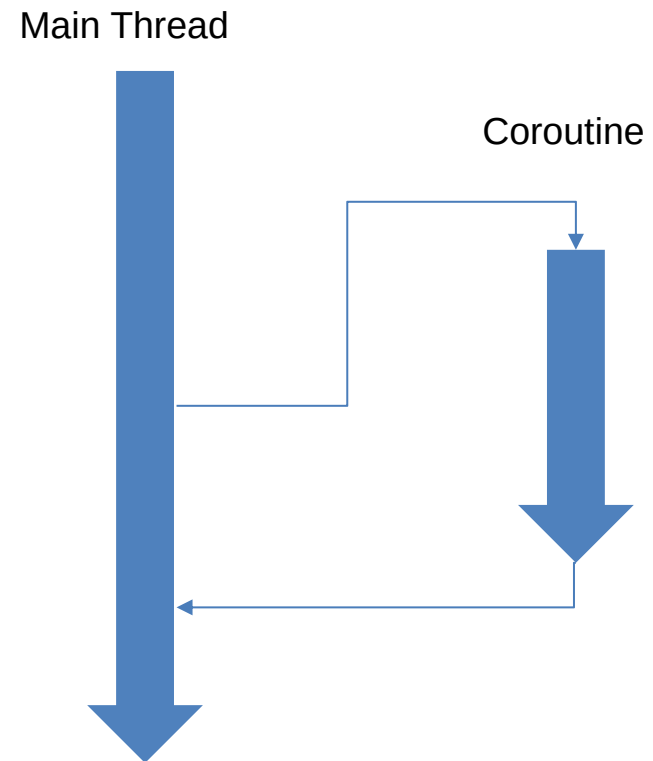
Coroutine

Asynchronous programming

Manage long-running tasks that might otherwise block the main thread and cause your app to freeze

Providing *main-safety*, or safely calling network or disk operations from the main thread

Coroutine



Coroutine

Two ways to start coroutine:

- **launch** starts a new coroutine and doesn't return the result to the caller. Any work that is considered "fire and forget" can be started using launch
- **async** starts a new coroutine and allows you to return a result with a suspend function called await

Coroutine

- There are 3 scopes in Kotlin coroutines:

	Global Scope	Life Cycle Scope	View Model Scope
Life time	Application	Activity	View Model
Resources	High	Medium	Low

Example

```
fun main() {  
    GlobalScope.launch { // launch a new coroutine in background and continue  
        delay(1000L)      // non-blocking delay for 1 second (default time unit is ms)  
        println("World!") // print after delay  
    }  
    println("Hello,")     // main thread continues while coroutine is delayed  
    Thread.sleep(2000L)   // block main thread for 2 seconds to keep JVM alive  
}
```

Output:

Hello,
World!

Question 4.1.5

1. What is the purpose of coroutine?
2. Why executing database-related operations on a main thread is not recommended?

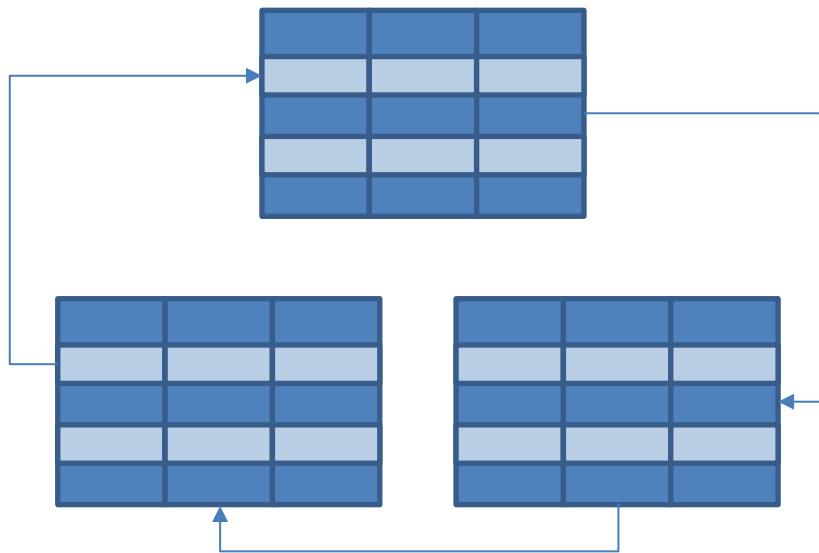


NoSQL - Firebase

SQL Database vs NoSQL Database

SQL

Table-based

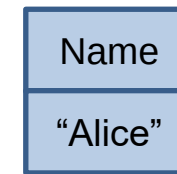


NoSQL

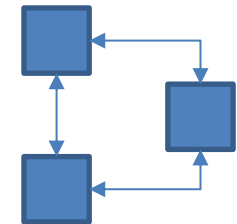
Document



Key-Value



Graph



Wide-Column Stores

Key	Name	Sex
1	Sam	M
2	Alice	F
3	John	M

Conclusion

Saving key-value pairs of simple data types in a shared preferences file

Saving arbitrary files in Android's file system

Using databases managed by SQLite

Using NoSQL - Firebase

ViewModel, Repository, LiveData and Coroutine